FPGA INT8 MLP ACCELERATOR

Design Specification

Architecture, arithmetic contract, and verification of the int8 MNIST accelerator

Team: Joey (`itsKannapi`), Teammate 2 (TBD), Teammate 3 (TBD) · Last updated 2026-10-22 · `https://github.com/itsKannapi/fpga-mlp-accelerator`

How to use this document

This is the *reference*: what the hardware does and why. If the RTL and this document disagree, one of them is a bug, so fix whichever is wrong in the same pull request. Day-to-day status lives in `progress.tex`; installing the tools lives in `setup.tex`.

Contents

1 Goal and scope **1**

2 Model and quantization **2**

3 Hardware architecture **4**

4 Verification **6**

5 Planned work and open design questions **9**

6 Repository layout **9**

1 Goal and scope

We are building a hardware inference engine for a small neural network that classifies handwritten digits (MNIST), running on a Digilent Basys 3 board (AMD Artix-7 `xc7a35tcpg236-1`, 100 MHz clock).

- **Network:** a multilayer perceptron, $784 \to 32 \to 10$, ReLU on the hidden layer. Trained in PyTorch, then quantized to 8-bit integers.
- **Hardware does inference only.** All training happens offline in Python. The FPGA runs the forward pass using fixed, pre-computed weights.
- **Correctness standard:** the hardware must match the Python integer model *bit for bit*, not approximately. Every intermediate value is checked against a golden reference (Section 4).
- **Deliverable:** classify MNIST test images sent from a PC over UART and report accuracy

measured on real hardware.

Explicitly out of scope for the first version: training on the FPGA, camera or other live input, convolutional layers, and external DDR memory (all weights fit on-chip, so none is needed).

2 Model and quantization

2.1 Training

`train_mlp.py` trains the float32 reference network.

Architecture	$784 \to 32$ (ReLU) $\to 10$, with biases
Optimizer	Adam, learning rate 10^{-3}, batch 128, 8 epochs, seed 0
Input scaling	pixels divided by 255 to $[0, 1]$; *no* mean/std normalization

Skipping mean/std normalization is deliberate: it keeps every input non-negative, so the hardware needs no signed input path. No softmax is implemented, because arg max gives the same answer with or without it.

2.2 Integer scheme

`quantize_export.py` converts the model to integers and **is the hardware specification**: whatever arithmetic it performs, the RTL must reproduce exactly. Quantization is symmetric and per-tensor (one scale per tensor rather than per neuron, which costs a little accuracy and saves a lot of hardware).

$$
\begin{aligned}
x_q &= \mathrm{round}(127\,x) \in [0, 127] && \text{input pixel (int8)} \\
W_q &= \mathrm{round}(W/s_W), \quad s_W = \max|W|/127 && \text{weights (int8)} \\
b_q &= \mathrm{round}\big(b/(s_x s_W)\big) && \text{biases (int32, accumulator scale)} \\
acc_1 &= \max\big(0,\ W_{1q}\,x_q + b_{1q}\big) && \text{layer 1 + ReLU (int32)} \\
h_q &= \min\big(127,\ (acc_1 + 2^{S-1}) \gg S\big), \quad S = 12 && \text{requantize: round half up, clamp} \\
acc_2 &= W_{2q}\,h_q + b_{2q} && \text{layer 2 (int32)} \\
\hat{y} &= \arg\max(acc_2) && \text{predicted digit}
\end{aligned}
$$

The shift amount S (`SHIFT1`) is the smallest shift that keeps the largest post-ReLU layer-1 value inside int8 range, measured on 2,000 *training* images. Calibrating on training images rather than test images keeps test data out of the design, so the reported accuracy is honest.

2.3 Measured results

float32 test accuracy (10,000 images)	95.86%
int8 test accuracy	95.86%
Accuracy change from quantization	+0.00 percentage points
Same prediction as float32	99.74% of images
`SHIFT1`	12 (rounding on)
Largest post-ReLU layer-1 value (calibration)	294,415
Accumulator headroom vs. int32	7,294×
Weight storage, layer 1 / layer 2	25,088 B / 320 B (24.8 KiB total)

Weights are frozen

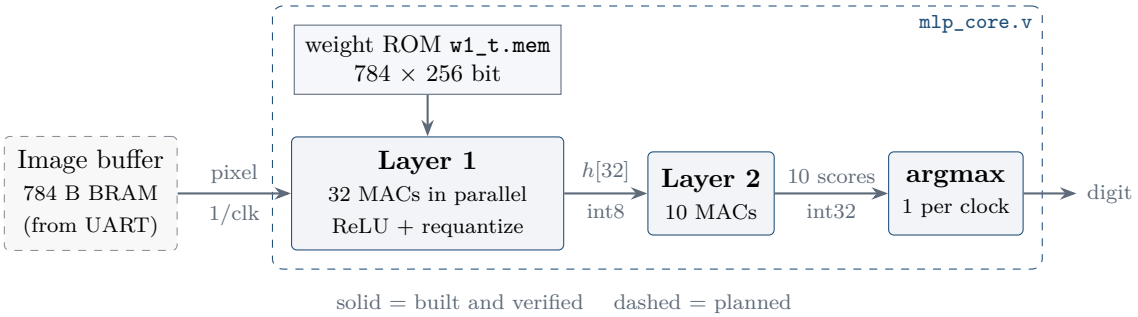
Training is not bit-reproducible across machines. The committed files in `rtl/` are the canonical weights; do not re-run `train_mlp.py` unless the team agrees to re-verify everything. `quantize_export.py` is deterministic given `mlp_fp32.npz`, but that file is not in git (see the open issue in `progress.tex`).

2.4 Generated files (`rtl/`)

File	Contents
`params.vh`	Verilog `` `define ``s: sizes, `ACC_W`, `SHIFT1`, `ROUND_SHIFT`. Single source of truth; never edit by hand.
`w1.mem`	Layer-1 weights, **neuron-major**: 784 weights for neuron 0, then neuron 1, … One int8 per line. Used by `tb_mac.v`.
`w1_t.mem`	Layer-1 weights, **pixel-major and packed**: 784 lines, each a 256-bit word holding pixel i's weight for all 32 neurons. Neuron n is bits $[8n{+}7{:}8n]$, so neuron 0 is the rightmost two hex digits. Read by `layer1.v`.
`b1.mem`, `b2.mem`	Biases, int32, two's complement hex.
`w2.mem`	Layer-2 weights, class-major (10×32). Used by `tb_layer2.v`.
`w2_t.mem`	Layer-2 weights, **hidden-major and packed**: 32 lines, each an 80-bit word holding hidden unit j's weight for all 10 classes; class n in bits $[8n{+}7{:}8n]$. Read by `layer2.v`.
`test_images.mem`, `test_labels.txt`	First 100 MNIST test images (int8) and labels, for the on-board run.
`golden.txt`	For the first 64 test images: label, prediction, and every intermediate value (`acc1`, `hq`, `acc2`). The testbenches compare against this.

3 Hardware architecture

3.1 Block diagram



3.2 MAC unit (`src/mac.v`)

One multiply-accumulate per clock: $acc \leftarrow acc + a \cdot b$.

Signal	Width	Behavior
`load`	1	At the clock edge, $acc \leftarrow$ `init` (the neuron's bias). **Has priority over `en`.**
`en`	1	At the clock edge, $acc \leftarrow acc + a \cdot b$. Otherwise holds.
`a`, `b`	8 signed	Activation and weight. The 16-bit product cannot overflow.
`init`, `acc`	32 signed	Bias in, running sum out.

The `(* use_dsp = "yes" *)` attribute maps each MAC onto one DSP48E1 slice. The Basys 3 has 90, which is the real limit on how many MACs can run in parallel.

3.3 Layer 1 (`src/layer1.v`)

All 32 hidden neurons are computed at once. Each clock, *one* pixel is broadcast to all 32 MACs, and one 256-bit read of the weight ROM gives every MAC its own weight for that pixel. Layer 1 therefore takes 784 accumulate cycles, regardless of the number of neurons (up to the DSP limit).

Port	Dir/width	Meaning
`start`	in, 1	Pulse for one cycle while idle to process an image. Ignored while busy.
`busy`	out, 1	High from the cycle after `start` until `done`.
`done`	out, 1	One-cycle pulse; `h_flat` is valid from then until the next start.
`pix_addr`	out, 10	Index (0–783) of the pixel wanted next.
`pix_data`	in, 8	That pixel, **valid one cycle after `pix_addr`** (synchronous BRAM timing).
`h_flat`	out, 256	32 int8 activations; neuron n in bits $[8n{+}7{:}8n]$.
`acc_flat`	out, 1024	Raw int32 accumulators (pre-ReLU), exposed for verification.

Timing. Address k is presented during the cycle after clock edge k.

Edge	Event
0	`start` sampled; FSM leaves idle
1	all MACs load their bias
$k+1$	pixel k and weight word k arrive from memory
$k+2$	MACs accumulate pixel k $\quad (k = 0 \ldots 783$, so edges 2–785)
786	requantized outputs registered; `done` goes high

Latency: 786 cycles = 7.86 µs at 100 MHz.

3.4 Layer 2 (`src/layer2.v`)

The same broadcast pattern at a smaller size: 10 MACs, one per class. Each clock one hidden value $h[j]$ goes to all 10 MACs, and an 80-bit word from `w2_t.mem` gives each its weight. The hidden value is registered on its way in, so it arrives with the same one-cycle latency a block-RAM pixel has in layer 1, and the control logic is cycle-for-cycle identical to layer 1 with 32 inputs instead of 784.

There is **no ReLU or requantize**: the 10 raw int32 scores go straight to argmax, which only compares them. Ports mirror layer 1: `start`, `busy`, `done`, `h_flat` in (must stay stable while busy), `scores_flat` out (class n in bits $[32n{+}31{:}32n]$). **Latency: 34 cycles.**

3.5 argmax (`src/argmax.v`)

Returns the index of the largest score, scanning one score per clock. A single-cycle 10-way comparison is a chain of nine 32-bit comparators, a likely timing problem at 100 MHz. The scan costs 10 cycles, about 1% of the total.

Tie-break: the *lowest* index wins, matching `numpy.argmax`, which produced the golden predictions. That comes from a strict `>` while scanning upward. `>=` would silently pick the highest index instead. Outputs `digit` (0–9) and `best` (the winning score). **Latency: 10 cycles.**

3.6 Full classifier (`src/mlp_core.v`)

Chains the three stages: each one starts on the previous stage's `done` pulse. The core accepts `start` only when the *whole chain* is idle. Layer 1 alone would accept a new image while layer 2 is still working, and its next result would then overwrite `h_flat` under layer 2. `busy` drops in the same cycle `done` pulses, so holding `start` high streams images back to back.

Latency, start to digit	832 cycles = 8.32 µs
layer 1 / handoff / layer 2 / handoff / argmax	786 / 1 / 34 / 1 / 10
Streaming throughput (start held high)	833 cycles per image ≈ 120,000 images/s
Accuracy, first 100 test images (simulated)	**99 / 100** (same as Python)

The one miss, test image 8 (a 5 classified as 6), is also misclassified by the Python model: the hardware reproduces the model, including its mistakes.

3.7 Resource estimate

Synthesized for Artix-7 with the open-source Yosys flow; Vivado's numbers will be recorded here once the board-level design is implemented.

Resource	Layer 1	Full core	Basys 3 has	Core share
DSP48E1 slices	32	42	90	47%
Block RAM (RAMB18)	15	15	100	15%
LUTs	1,657	2,162	20,800	10%
Flip-flops	1,299	1,408	41,600	3%

Layer 2's 32 × 80-bit weights are small enough to live in LUTs, so the block-RAM count does not change.

4 Verification

4.1 Strategy

1. Python produces **golden vectors**: the exact integer value of every intermediate signal for 64 real test images (`golden.txt`).
2. Each testbench runs the RTL on the same inputs and compares every value. Python and Verilog are independent implementations, so agreement means the hardware matches the model.
3. Every testbench is **self-checking**: it prints `ALL TESTS PASSED` or names the exact failing value. Nobody has to read waveforms to know whether it worked.
4. Every testbench is checked by **mutation testing**: we plant realistic bugs in the RTL and confirm the testbench catches each one. A testbench that passes is only meaningful if it can also fail.

4.2 Testbench results

Testbench	What it checks	Checks	Status
`tb_mac.v`	10 directed edge cases (sign, extremes, hold, load priority); 2,000 random dot products; 8 images × 32 neurons vs. golden	2,266	DONE
`tb_layer1.v`	64 golden images × 32 neurons, both `acc1` and `hq`; 3 synthetic images; latency, `busy`, `done`, and a stray `start` on every run	4,259	DONE
`tb_layer2.v`	layer 2 alone, fed the golden hidden values: 64 images × 10 scores; 3 synthetic inputs; control	737	DONE
`tb_argmax.v`	12 directed cases (ties, int32 extremes, signed trap); 2,000 random vectors, 526 with a tied maximum; control	4,024	DONE
`tb_mlp_core.v`	end to end, pixels in: hidden values, scores, and digit for 64 golden images; stray starts during each stage; 100 images streamed back to back; accuracy	2,880	DONE
Total		**14,166**	

All pass in Icarus Verilog (`scripts/run_sims.sh`). `tb_mac`, `tb_layer1`, and the end-to-end `tb_mlp_core` are also confirmed in Vivado's simulator (xsim).

4.3 Why `tb_layer1` needs synthetic images

Mutation testing showed that real MNIST digits have blind spots. Their border pixels are always black, so dropping the first or last pixel changes no result. No real image pushes a hidden neuron past 127, so a missing clamp is invisible. Three made-up images close those gaps: a *ramp* (every pixel nonzero and different), *all-127*, and a *hot* image that lights every pixel where the strongest neuron has a positive weight. Their expected values are computed from `w1.mem` (neuron-major), not the `w1_t.mem` the hardware reads, so a packing error in `w1_t.mem` is caught too. The test prints how many times ReLU (41) and the clamp (2) were exercised, as proof those paths were tested.

4.4 Mutation testing results

`tb_layer1.v`: number of failing checks per planted bug. Every bug is caught.

Planted bug	acc1	hq	synthetic	control	
MAC enable driven a cycle early (pipeline slip)	0	0	52	0	caught
Weight memory one address behind pixels	2048	2048	96	0	caught
Truncate instead of round when requantizing	0	705	24	0	caught
Weight bytes in reversed order	1875	1811	96	0	caught
Bias not reloaded between images	2048	2048	96	0	caught
ReLU missing before requantize	0	583	41	0	caught
Last pixel skipped	0	0	52	67	caught
First pixel skipped	0	0	87	67	caught
Clamp to 127 missing	0	0	2	0	caught
Shift off by one	0	1437	53	0	caught

Note the first row: a pipeline-timing bug that passes *every* real image and is caught only by the synthetic stage.

`tb_mac.v`: unsigned multiply (2,261 failures), `en` overriding `load` (caught only by the directed test), and a 16-bit truncated accumulator (caught only by the random and golden tests). Each tier catches bugs the others miss.

Layer 2, argmax, and the core: failing checks in the module's own testbench and in the end-to-end `tb_mlp_core.v`.

Module	Planted bug	Unit test	End-to-end
argmax	`>=` instead of `>` (highest index wins ties)	531	**0 (missed)**
argmax	scan stops before class 9	2,164	78
argmax	unsigned comparison	1,993	128
argmax	running maximum starts at 0, not score 0	279	4
layer 2	last hidden value skipped	727	706
layer 2	first hidden value skipped	487	466
layer 2	weight bytes in reversed order	670	768
layer 2	bias not reloaded between images	670	756
layer 2	MAC enable a cycle early	660	642
core	start accepted while layer 2 / argmax busy	–	2
core	argmax started by layer 1 instead of layer 2	–	769
core	`busy` still high during `done`	–	64

Every bug is caught somewhere. The first row is why `tb_argmax` exists: real MNIST scores never tie, so a wrong tie-break is invisible end to end. Only a directed tie test catches it. The first "core" bug was at first caught only by luck (it showed up as a wrong latency on the *next* image). The

testbench now watches every stage for 1,000 cycles after each stray start, so it is caught where it happens.

5 Planned work and open design questions

- **Image buffer + UART**: 784-byte BRAM written by a UART receiver; prediction sent back by a UART transmitter. Protocol not yet defined. The host can send raw MNIST bytes and the board converts each with `pixel` $\gg 1$. This was checked: for all 256 byte values it gives exactly the same int8 as `quantize_input` in Python.
- **Board top level**: `mlp_core` plus image buffer, UART, the Basys 3 clock and reset, and a 7-segment display of the predicted digit.
- **Timing closure** at 100 MHz once the board-level design is assembled. The longest paths are likely the 32 parallel requantizers in layer 1; they can be pipelined if needed.
- **Parameterize the layers**: `layer1.v` and `layer2.v` share their control logic and could become one parameterized module.

6 Repository layout

`src/`	Synthesizable Verilog: `mac.v`, `layer1.v`, `layer2.v`, `argmax.v`, `mlp_core.v` (top)
`sim/`	Self-checking testbenches (`tb_*.v`)
`rtl/`	Generated weights, golden vectors, `params.vh`. Committed; canonical.
`scripts/`	`create_project.tcl` (rebuilds the Vivado project), `run_sims.sh` (runs every testbench in Icarus)
`docs/`	This document, `progress.tex`, `setup.tex`, and their PDFs
`vivado/`	Generated Vivado project. **Not committed**; rebuilt per machine.
`*.py`	`train_mlp.py` (training), `quantize_export.py` (the spec)